

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy Sierra: Head First Design Patterns,
O'Reilly Media 2020



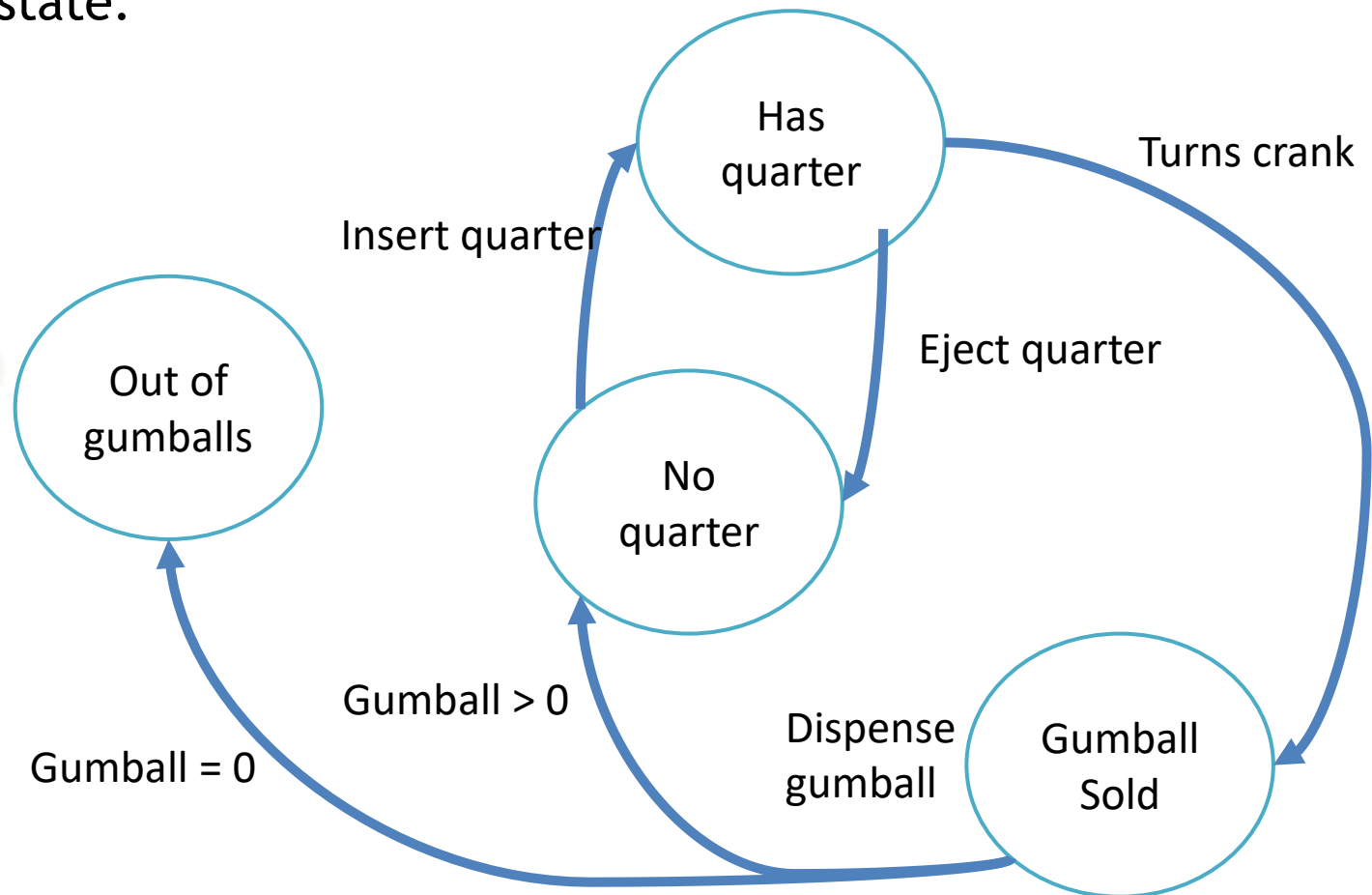
Design Pattern
State and Proxy



Francesco Guerra
francesco.guerra@unimore.it

The State Pattern

- State helps objects to control their behavior by changing their internal state.



Creating the state machine

- ▶ Considering 4 states and an instance variable to hold the current state

```
final static int SOLD_OUT = 0;
final static int NO_QUARTER = 1;
final static int HAS_QUARTER = 2;
final static int SOLD = 3;
int state = SOLD_OUT;
```

- ▶ We create a class that acts as the state machine.
 - ▶ For each action (insert quarter, eject quarter ...), there is method that uses conditional statements to determine what behavior is appropriate.

```
public void insertQuarter() {
    if (state == HAS_QUARTER) {
        System.out.println("You can't insert another quarter");
    } else if (state == NO_QUARTER) {
        state = HAS_QUARTER;
        System.out.println("You inserted a quarter");
    } else if (state == SOLD_OUT) {
        System.out.println("You can't insert a quarter, machine sold out");
    } else if (state == SOLD) {
        System.out.println("Please wait, we're giving you a gumball");
    }
}
```

Creating the state machine (2)

- ▶ We are
 - ▶ modeling state within an object by creating an instance variable to hold the state values
 - ▶ writing conditional code within our methods to handle the various states.

The (partial) code

```
public class GumballMachine {
    final static int SOLD_OUT = 0;
    final static int NO_QUARTER = 1;
    final static int HAS_QUARTER = 2;
    final static int SOLD = 3;
    int state = SOLD_OUT;
    int count = 0;
    public GumballMachine(int count) {
        this.count = count;
        if (count > 0) {
            state = NO_QUARTER;
        }
    }
    public void insertQuarter() {
        if (state == HAS_QUARTER) {
            System.out.println("You can't insert another quarter");
        } else if (state == NO_QUARTER) {
            state = HAS_QUARTER;
            System.out.println("You inserted a quarter");
        } else if (state == SOLD_OUT) {
            System.out.println("The machine is sold out");
        } else if (state == SOLD) {
            System.out.println("We're already giving you a gumball");
        }
    }
}
```



```
public class GumballMachineTestDrive {  
    public static void main(String[] args) {  
        GumballMachine gumballMachine = new GumballMachine(5);  
        System.out.println(gumballMachine);
```

Testing

```
gumballMachine.insertQuarter();  
gumballMachine.turnCrank();  
System.out.println(gumballMachine);
```

```
gumballMachine.insertQuarter();  
gumballMachine.ejectQuarter();  
gumballMachine.turnCrank();  
System.out.println(gumballMachine);
```

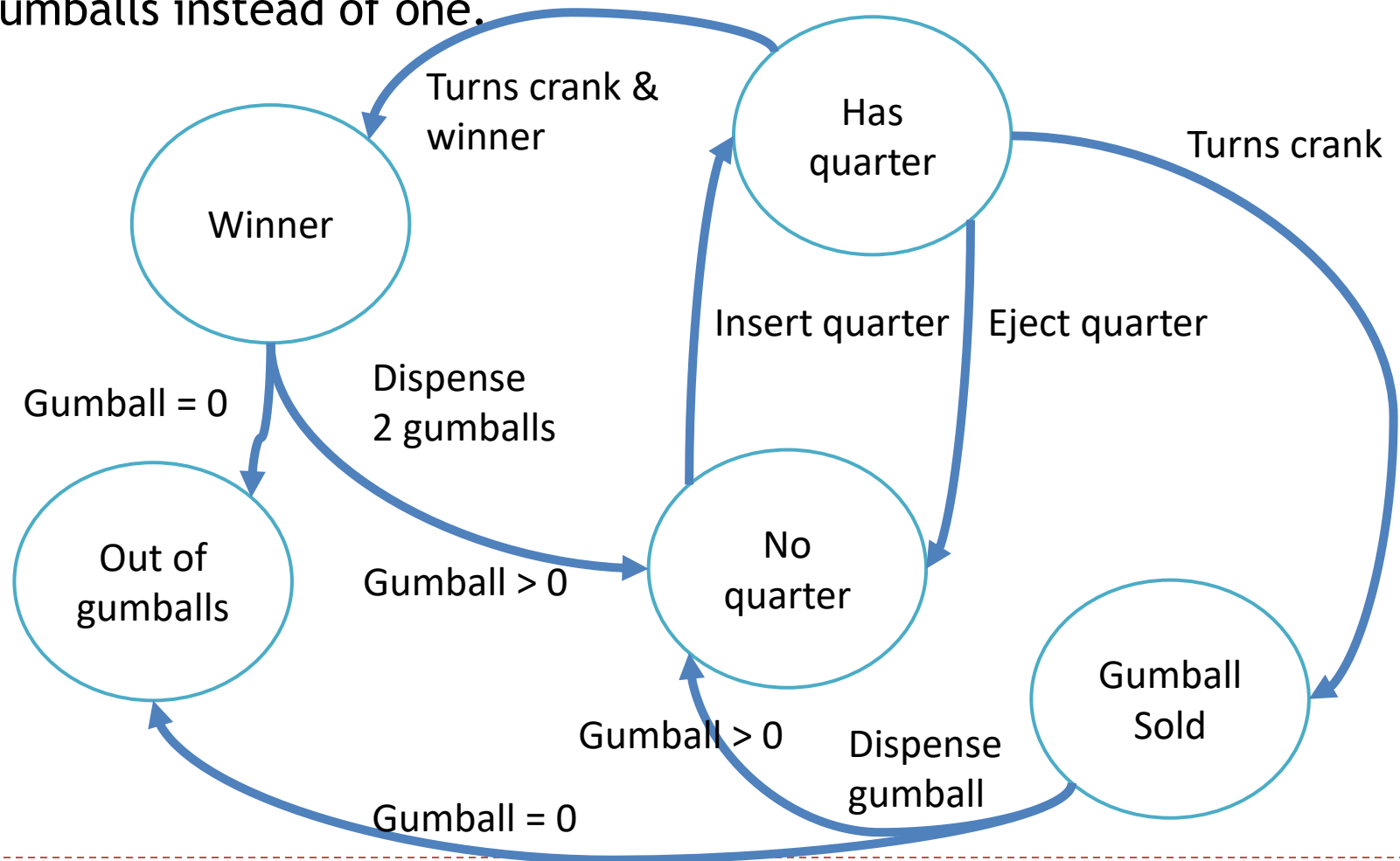
```
gumballMachine.insertQuarter();  
gumballMachine.turnCrank();  
gumballMachine.insertQuarter();  
gumballMachine.turnCrank();  
gumballMachine.ejectQuarter();  
System.out.println(gumballMachine);
```

```
gumballMachine.insertQuarter();  
gumballMachine.insertQuarter();  
gumballMachine.turnCrank();  
gumballMachine.insertQuarter();  
gumballMachine.turnCrank();  
gumballMachine.insertQuarter();  
gumballMachine.turnCrank();  
System.out.println(gumballMachine);
```

The test includes some stress testing operations

Improvement

- ▶ 10% of the time, when the crank is turned, the customer gets two gumballs instead of one.



Implementation

```
final static int SOLD_OUT = 0;
final static int NO_QUARTER = 1;
final static int HAS_QUARTER = 2;
final static int SOLD = 3;
-- public void insertQuarter() {
    // insert quarter code here
}
public void ejectQuarter() {
    // eject quarter code here
}
public void turnCrank() {
    // turn crank code here
}
public void dispense() {
    // dispense code here
}
```

- ▶ The code previously implemented is not simple to extend
 - ▶ You'd have to add a new WINNER state
 - ▶ You'd have to add a new conditional in every single method to handle the WINNER state
 - ▶ that's a lot of code to modify
 - ▶ turnCrank() will get especially messy: you'd have to add code to see whether you've got a WINNER and then switch to either the WINNER state or the SOLD state.

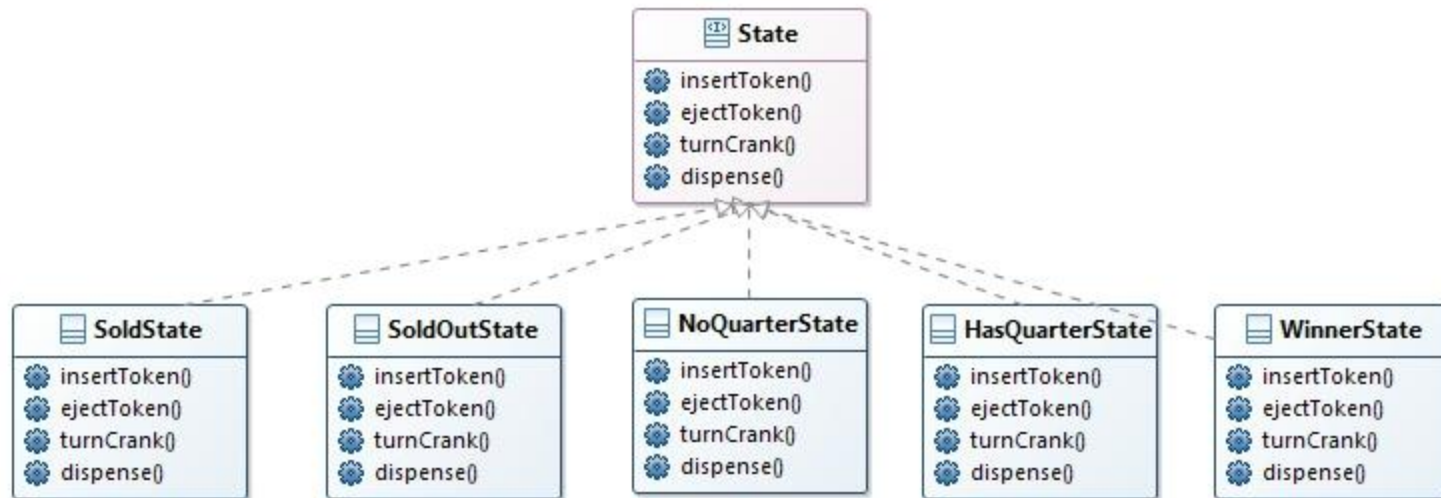
Implementation (2)

- ▶ The current implementation
 - ▶ This code certainly isn't adhering to the Open Closed Principle.
 - ▶ This design isn't even very object-oriented.
 - ▶ State transitions aren't explicit; they are buried in the middle of a bunch of conditional statements.
 - ▶ We haven't encapsulated anything that varies here.
 - ▶ Further additions are likely to cause bugs in working code

The new design

- ▶ A solution could be to rework the existing code and
 - ▶ encapsulate state objects in their own classes
 - ▶ delegate to the current state when an action occurs.
- ▶ Steps
 - ▶ define a State interface that contains a method for every action in the Gumball Machine.
 - ▶ implement a State class for every state of the machine.
 - ▶ These classes will be responsible for the behavior of the machine when it is in the corresponding state.
 - ▶ delegate to the State class to do the work for us.

The UML representation



State will be an attribute of the GumballMachine.

Some methods will allow to change the "state" associated to that class

Implementing the State Classes

```
public class NoQuarterState implements State {
    GumballMachine gumballMachine;

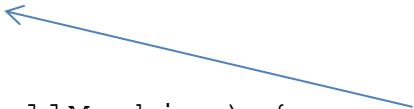
    public NoQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You inserted a quarter");
        gumballMachine.setState(gumballMachine.getHasQuarterState());
    }

    public void ejectQuarter() {
        System.out.println("You haven't inserted a quarter");
    }

    public void turnCrank() {
        System.out.println("You turned, but there's no quarter");
    }

    public void dispense() {
        System.out.println("You need to pay first");
    }
}
```



**We need to implement
the interface**

```
public class GumballMachine {
```

```
    State soldOutState;
    State noQuarterState;
    State hasQuarterState;
    State soldState;
    State state;
    int count = 0;
```

```
    public GumballMachine(int numberGumballs) {
        soldOutState = new SoldOutState(this);
        noQuarterState = new NoQuarterState(this);
        hasQuarterState = new HasQuarterState(this);
        soldState = new SoldState(this);
        this.count = numberGumballs;
        if (numberGumballs > 0) {
            state = noQuarterState;
        } else {
            state = soldOutState;
        }
    }
```

```
    public void insertQuarter() {
        state.insertQuarter();
    }
    public void ejectQuarter() {
        state.ejectQuarter();
    }
    public void turnCrank() {
        state.turnCrank();
        state.dispense();
    }
}
```

```
    void setState(State state) {
        this.state = state;
    }
```

```
    void releaseBall() {
        System.out.println("A gumball
comes rolling out the slot...");
        if (count != 0) {
            count = count - 1;
        }
    }
    // More methods here including
    getters for each State...
}
```

The machine

Implementing more states

```
public class HasQuarterState implements State {
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }

    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }

    public void turnCrank() {
        System.out.println("You turned...");
        gumballMachine.setState(gumballMachine.getSoldState());
    }

    public void dispense() {
        System.out.println("No gumball dispensed");
    }
}
```



ISSUE: Look at the GumballMachine implementation.

If the crank is turned and not successful (say the customer didn't insert a quarter first), we call `dispense` anyway, even though it's unnecessary.

Implementing more states (2)

NOW, LET'S CHECK OUT THE `SoldState` CLASS...

```
public class SoldState implements State {
    //constructor and instance variables here

    public void insertQuarter() {
        System.out.println("Please wait, we're already giving you a gumball");
    }

    public void ejectQuarter() {
        System.out.println("Sorry, you already turned the crank");
    }

    public void turnCrank() {
        System.out.println("Turning twice doesn't get you another gumball!");
    }

    public void dispense() {
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() > 0) {
            gumballMachine.setState(gumballMachine.getNoQuarterState());
        } else {
            System.out.println("Oops, out of gumballs!");
            gumballMachine.setState(gumballMachine.getSoldOutState());
        }
    }
}
```

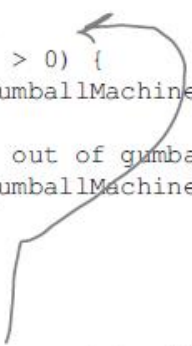
Here are all the inappropriate actions for this state



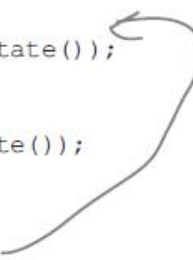
And here's where the real work begins...



We're in the `SoldState`, which means the customer paid. So, we first need to ask the machine to release a gumball.



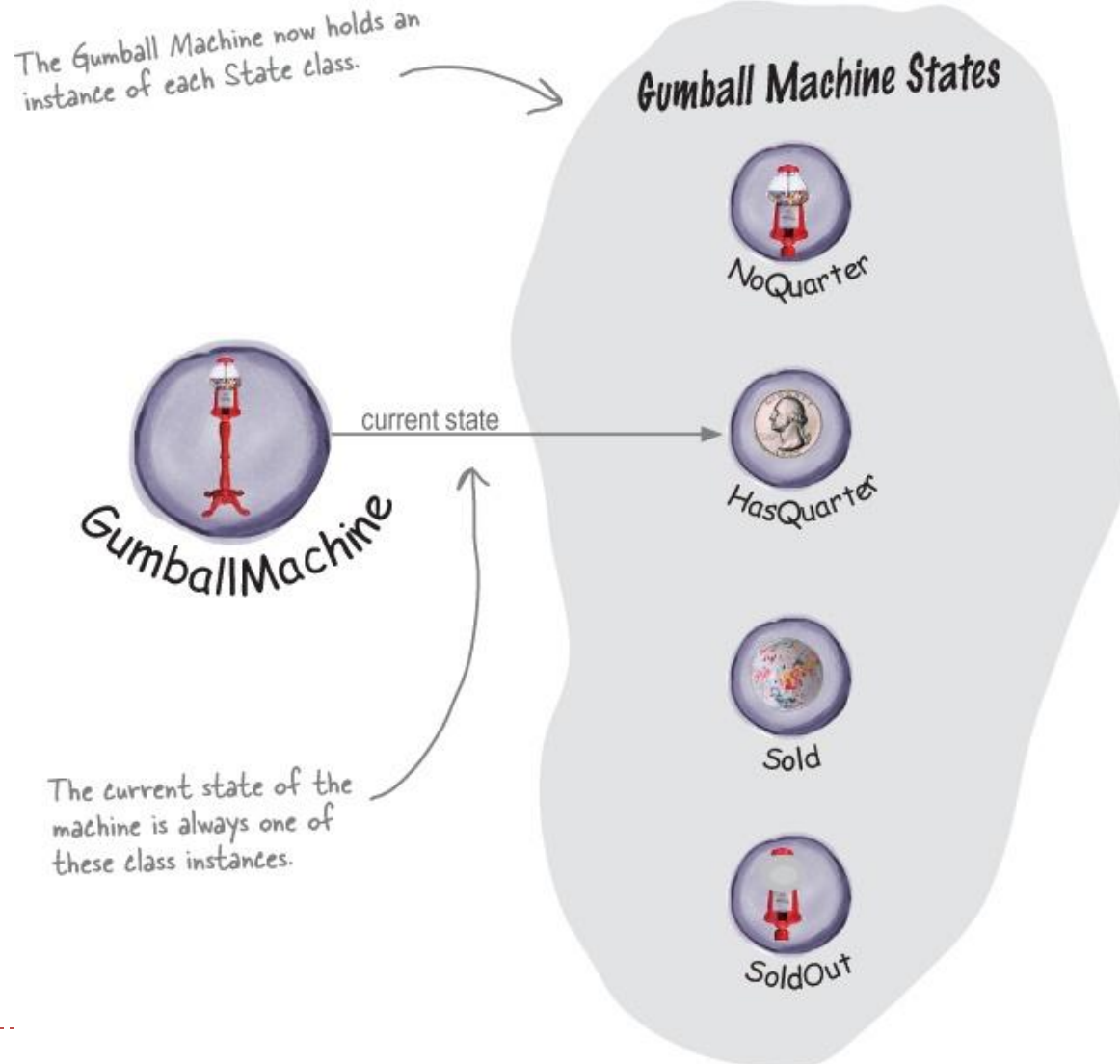
Then we ask the machine what the gumball count is, and either transition to the `NoQuarterState` or the `SoldOutState`.



The new design

- ▶ The Gumball Machine implementation is structurally different from the first version, and yet functionally it is exactly the same.
- ▶ There is
 - ▶ the behavior of each state localized into its own class.
 - ▶ All the troublesome if statements that would have been difficult to maintain have been removed.
 - ▶ Closed each state for modification, and yet left the Gumball Machine open to extension.
 - ▶ Created a code base and class structure that maps much more closely to the diagram and is easier to read and understand.

A picture of the new design



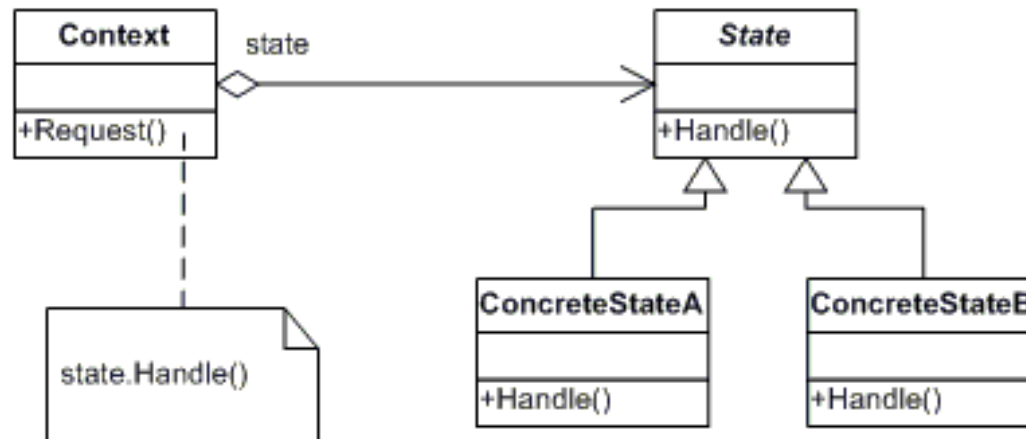
The State Pattern defined

- ▶ The State Pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class.
- ▶ Because the pattern encapsulates state into separate classes and delegates to the object representing the current state, we know that behavior changes along with the internal state.
- ▶ What does it mean for an object to “appear to change its class”?
 - ▶ Think about it from the perspective of a client:
 - ▶ if an object you’re using can completely change its behavior, then it appears to you that the object is actually instantiated from another class.
 - ▶ In reality, however, you know that we are using composition to give the appearance of a class change by simply referencing different state objects.

State Pattern class diagram

The Context is the class that can have a number of internal states. In our example, the GumballMachine is the Context.

The State interface defines a common interface for all concrete states; the states all implement the same interface, so they are interchangeable.



Whenever the `request()` is made on the Context it is delegated to the state to handle.

ConcreteStates handle requests from the Context. Each ConcreteState provides its own implementation for a request. In this way, when the Context changes state, its behavior will change as well.

Many concrete states are possible

State vs Strategy Pattern

- ▶ The class diagrams are the same, but the two patterns differ in their intent.
- ▶ With the State Pattern,
 - ▶ we have a set of behaviors encapsulated in state objects;
 - ▶ at any time the context is delegating to one of those states.
 - ▶ Over time, the current state changes across the set of state objects to reflect the internal state of the context, so the context's behavior changes over time as well.
 - ▶ The client usually knows very little, if anything, about the state objects.
- ▶ With Strategy,
 - ▶ the client usually specifies the strategy object that the context is composed with.
 - ▶ Often there is a strategy object that is most appropriate for a context object.
- ▶ Think of the Strategy Pattern as a flexible alternative to subclassing.
- ▶ Think of the State Pattern as an alternative to putting lots of conditionals in your context.

Questions / Comments

- ▶ In the GumballMachine, the states decide what the next state should be. Do the Concrete States always decide what state to go to next?
 - ▶ The alternative is to let the Context decide on the flow of state transitions.
 - ▶ As a general guideline, when the state transitions are fixed they are appropriate for putting in the Context; however, when the transitions are more dynamic, they are typically placed in the state classes themselves.
 - ▶ The disadvantage of having state transitions in the state classes is that we create dependencies between the state classes. In our implementation of the GumballMachine we tried to minimize this by using getter methods on the Context, rather than hardcoding explicit concrete state classes.
- ▶ If I have lots of instances of the Context in my application, is it possible to share the state objects across them?
 - ▶ This is a very common scenario.
 - ▶ The only requirement is that your state objects do not keep their own internal context; otherwise, you'd need a unique instance per context.
 - ▶ To share your states, you'll typically assign each state to a static instance variable. If your state needs to make use of methods or instance variables in your Context, you'll also have to give it a reference to the Context in each handler() method.

Questions / Comments (2)

- ▶ It seems like using the State Pattern increases the number of classes in our designs.
 - ▶ By encapsulating state behavior into separate state classes, you'll always end up with more classes in your design.
 - ▶ That's often the price you pay for flexibility.
 - ▶ If you have an application that has a lot of state and you decide not to use separate objects, you'll instead end up with very large, monolithic conditional statements. This makes your code hard to maintain and understand. By using objects, you make states explicit and reduce the effort needed to understand and maintain your code.
- ▶ The State Pattern class diagram shows that State is an abstract class. But didn't you use an interface in the implementation of the gumball machine's state?
 - ▶ Yes. Given we had no common functionality to put into an abstract class, we went with an interface. In your own implementation, you might want to consider an abstract class. Doing so has the benefit of allowing you to add methods to the abstract class later, without breaking the concrete state implementations.

The new Gumball machine with the winner state

```
public class WinnerState
implements State {
```

```
    //
    // insertQuarter error message
    // ejectQuarter error message
    // turnCrank error message
```

```
    public void dispense() {
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() == 0) {
            gumballMachine.setState(gumballMachine.getSoldOutState());
        } else {
            gumballMachine.releaseBall();
            System.out.println("YOU'RE A WINNER! You got two gumballs");
            if (gumballMachine.getCount() > 0) {
                gumballMachine.setState(gumballMachine.getNoQuarterState());
            } else {
                System.out.println("Oops, out of gumballs!");
                gumballMachine.setState(gumballMachine.getSoldOutState());
            }
        }
    }
}
```

```
public class GumballMachine {
    State soldOutState;
    State noQuarterState;
    State hasQuarterState;
    State soldState;
    State winnerState;

    State state = soldOutState;
    int count = 0;
    // methods here
}
```

The HasQuarterState class modified

```
public class HasQuarterState implements State {
    Random randomWinner = new Random(System.currentTimeMillis());
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }

    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }

    public void turnCrank() {
        System.out.println("You turned...");
        int winner = randomWinner.nextInt(10);
        if ((winner == 0) && (gumballMachine.getCount() > 1)) {
            gumballMachine.setState(gumballMachine.getWinnerState());
        } else {
            gumballMachine.setState(gumballMachine.getSoldState());
        }
    }

    public void dispense() {System.out.println("No gumball dispensed");}
```

Winner State

- ▶ Why do we need the WinnerState? Couldn't we just have the SoldState dispense two gumballs?
- ▶ SoldState and WinnerState are almost identical, except that WinnerState dispenses two gumballs instead of one.
- ▶ You certainly could put the code to dispense two gumballs into the SoldState.
- ▶ The downside is that now you've got TWO states represented in one State class: the state in which you're a winner, and the state in which you're not.
- ▶ So you are sacrificing clarity in your State class to reduce code duplication.
- ▶ Another thing to consider is the principle One class, One responsibility.
- ▶ By putting the WinnerState responsibility into the SoldState, you've just given the SoldState TWO responsibilities.
- ▶ What happens when the promotion ends? Or the stakes of the contest change? So, it's a tradeoff and comes down to a design decision.

Issues

- ▶ We've got a lot of duplicate code in the Sold and Winning states and we might want to clean those up.
 - ▶ We could make State into an abstract class and build in some default behavior for the methods with the error messages.
- ▶ The dispense() method always gets called, even if the crank is turned when there is no quarter.
 - ▶ We could easily fix this by having turnCrank() return a boolean, or by introducing exceptions. Which do you think is a better solution?
- ▶ Will you be instantiating a lot of GumballMachine objects?
 - ▶ If so, you may want to move the state instances into static instance variables and share them.

Key points

- ▶ The State Pattern allows an object to have many different behaviors based on its internal state.
 - ▶ The Context gets its behavior by delegating to the current state object it is composed with.
 - ▶ By encapsulating each state into a class, we localize any changes that will need to be made.
- ▶ The State and Strategy Patterns have the same class diagram, but they differ in intent.
 - ▶ Strategy Pattern typically configures Context classes with a behavior or algorithm. □
 - ▶ State Pattern allows a Context to change its behavior as the state of the Context changes.
- ▶ State transitions can be controlled by the State classes or by the Context classes.
- ▶ Using the State Pattern will typically result in a greater number of classes in your design.
- ▶ State classes may be shared among Context instances.

The Proxy Pattern

- ▶ The Proxy pattern controls and manages access.
 - ▶ Let us to implement a code for monitoring the gumball machine.

```
public class GumballMonitor {
    GumballMachine machine;
    public GumballMonitor(GumballMachine machine) { this.machine = machine;}
    public void report() {
        System.out.println("Gumball Machine: " + machine.getLocation());
        System.out.println("Current inventory: " + machine.getCount() + "
gumballs");
        System.out.println("Current state: " + machine.getState());
    }
}

public class GumballMachine {
    // other instance variables
    String location;
    public GumballMachine(String location, int count) {
        // other constructor code here
        this.location = location;
    }
    public String getLocation() {
        return location;
    }
    // other methods here
}
```

Testing the Monitor

```
public class GumballMachineTestDrive {
    public static void main(String[] args) {
        int count = 0;
        if (args.length < 2) {
            System.out.println("GumballMachine <name> <inventory>");
            System.exit(1);
        }
        count = Integer.parseInt(args[1]);
        GumballMachine gumballMachine = new GumballMachine(args[0], count);
        GumballMonitor monitor = new GumballMonitor(gumballMachine);

        // rest of test code here
        monitor.report();
    }
}
```

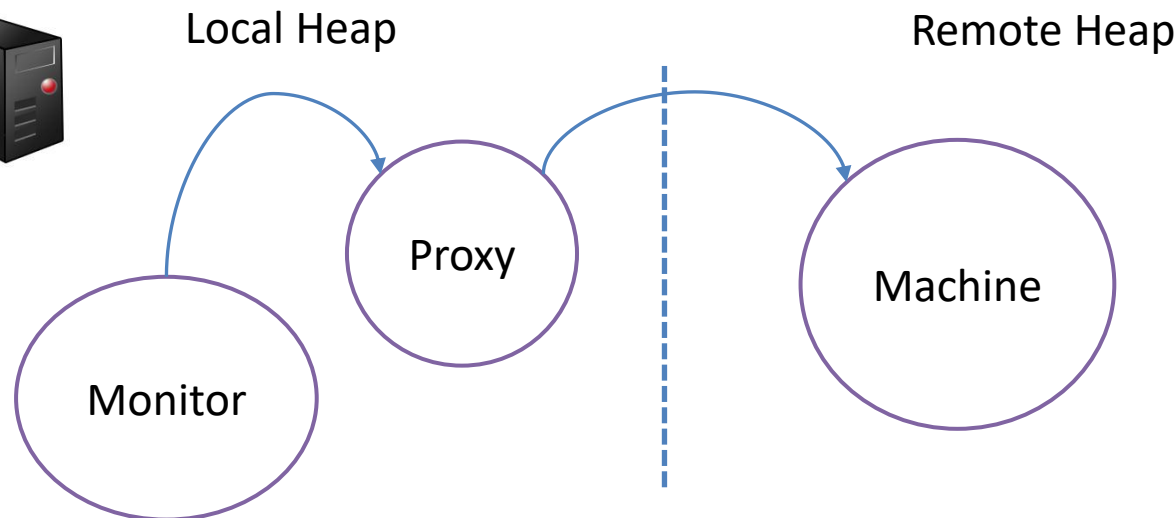
- ▶ The monitor looks good, but typically we have to monitor devices remotely.

A remote proxy

- ▶ In the current implementation we give the GumballMonitor a reference to a machine and it gives us a report.
 - ▶ The problem is that the monitor runs in the same JVM as the gumball machine
- ▶ So what if we left our GumballMonitor class as is, but handed it a proxy to a remote object?
 - ▶ A proxy is a stand in for a real object.
 - ▶ In this case, the proxy acts just like it is a Gumball Machine object, but behind the scenes it is communicating over the network to talk to the real, remote GumballMachine.

The role of the 'remote proxy'

- ▶ A remote proxy acts as a local representative to a remote object.
 - ▶ What's a "remote object"?
 - ▶ It's an object that lives in the heap of a different Java Virtual Machine
 - ▶ What's a "local representative"?
 - ▶ It's an object that you can call local methods on and have them forwarded on to the remote object.



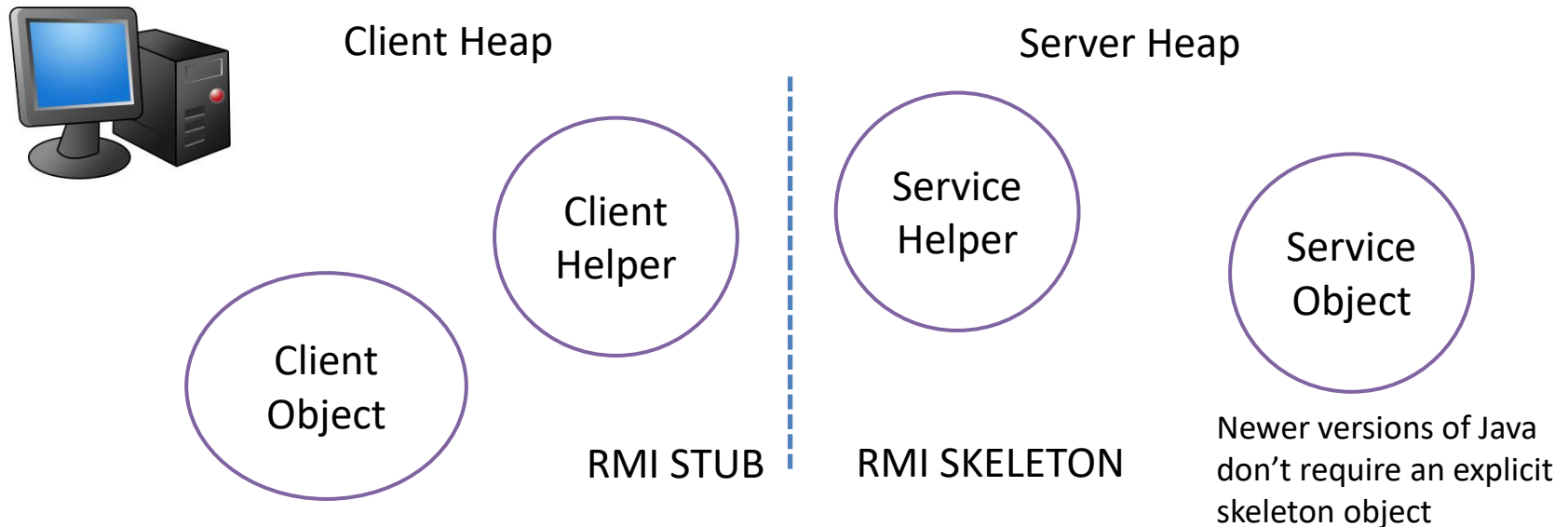
What we need?

- ▶ We need to write some code that
 - ▶ takes a method invocation,
 - ▶ somehow transfers it over the network,
 - ▶ invokes the same method on a remote object.
- ▶ Then I presume when the call is complete, the result gets sent back over the network to our client.
- ▶ it's pretty much built into Java's remote invocation functionality.
- ▶ All we have to do is retrofit our code so that it takes advantage of RMI

Remote methods

- ▶ **Goal:** allow a local object to forward calls to a remote object.
 - ▶ Use *helper objects* to manage all communication between client and server.
- ▶ **Client Side**
 - ▶ The client helper acts as a proxy for the remote service.
 - ▶ The client calls it as if it were a local object.
- ▶ **The helper:**
 - ▶ Sends the method call and parameters to the server.
 - ▶ Waits for the response.
- ▶ **Server Side**
 - ▶ The service helper receives the request (via socket).
 - ▶ Unpacks the call information.
 - ▶ Invokes the real method on the service object.
 - ▶ Sends the result back to the client helper.

Java RMI



- ▶ What RMI does is to build the client and service helper objects, creating a client helper object with the same methods as the remote service.
 - ▶ You don't have to write any of the networking or I/O code yourself.
- ▶ There is one difference between RMI calls and local (normal) method calls.
 - ▶ They can fail due to the network connection.
- ▶ RMI provides all the runtime infrastructure to make it all work, including a lookup service that the client can use to find and access the remote objects.

Making the Remote service

- ▶ Step one: Make the remote interface
 - ▶ The remote interface defines the methods that a client can call remotely. It's what the client will use as the class type for your service.
- ▶ Step two: Make a remote implementation
 - ▶ This is the class that does the Real Work. It has the real implementation of the remote methods defined in the remote interface. It's the object that the client wants to call methods on.
- ▶ Step three: Start the RMI registry
 - ▶ The rmiregistry is like the white pages of a phone book. It's where the client goes to get the proxy (the client stub/helper object)
- ▶ Step Four: Start the remote service
 - ▶ You have to get the service object up and running. Your service implementation class instantiates an instance of the service and registers it with the RMI registry. Registering it makes the service available for clients.

Step one: make a Remote interface

```
import java.rmi.*;
public interface MyRemote extends Remote {
    public String sayHello() throws RemoteException;
}
```

- ▶ Remote is a ‘marker’ interface: it has no methods.
 - ▶ Notice that we say ‘extends’ here. One interface is allowed to extend another interface.
- ▶ Declare that all methods throw a RemoteException
 - ▶ The remote interface is the one the client uses as the type for the service.
 - ▶ In other words, the client invokes methods on something that implements the remote interface.
- ▶ Be sure arguments and return values are primitives or Serializable
 - ▶ Arguments and return values of a remote method must be either primitive or Serializable.

Java Serializable

- ▶ A class is **serializable** if it implements the `java.io.Serializable` interface.
- ▶ This enables objects to be **converted into a byte stream** and later **reconstructed** (deserialization).
- ▶ Classes that do **not** implement `Serializable` cannot have their state saved or restored.
- ▶ All **subclasses** of a serializable class are automatically serializable.
- ▶ The `Serializable` interface has **no methods or fields**: it simply marks a class as serializable.
 - ▶ Classes can customize serialization with `writeObject()` and `readObject()` methods.

Step two: make a Remote implementation

- ▶ Implement the Remote Interface
 - ▶ The service has to implement the remote interface, with the methods your client will going to call
- ▶ Extend UnicastRemoteObject
 - ▶ To work as a remote service object, your object needs some functionality. The simplest way is to extend UnicastRemoteObject (from the java.rmi.server package) and let that class (your superclass) do the work for you
- ▶ Write a constructor that declares a RemoteException
 - ▶ UnicastRemoteObject constructor throws a RemoteException. The only way to deal with this is to declare a constructor for your remote implementation, just so that you have a place to declare the RemoteException. When a class is instantiated, its superclass constructor is always called. If your superclass constructor throws an exception, you have no choice but to declare that your constructor also throws an exception.
- ▶ Register the service with the RMI registry
 - ▶ This way you make it available to remote clients

Step two: make a Remote implementation

```
import java.rmi.*;
import java.rmi.server.*;
public class MyRemoteImpl extends
    UnicastRemoteObject implements MyRemote {
    private static final long serialVersionUID = 1L;
    public String sayHello() {
        return "Server says, 'Hey'";
    }
    public MyRemoteImpl() throws RemoteException { }
    public static void main (String[] args) {
        try {
            MyRemote service = new MyRemoteImpl();
            Naming.rebind("rmi://localhost:1999/RemoteHello", service);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

The service has to implement the remote interface with the methods your client is going to call

You have to make the service available to remote clients, by instantiating it and putting it into the RMI registry . When you register the implementation object, the RMI system actually puts the stub in the registry, since that's what the client really needs. Register your service using the static rebind() method of the java.rmi.Naming class.

make a Remote implementation (2)

```
%rmiregistry  
%java MyRemoteImpl
```

Recall: rmiregistry should starts in the same directory as the rmi Server

- ▶ Step three: run rmiregistry
 - ▶ Bring up a terminal and start the rmiregistry.
 - ▶ Be sure you start it from a directory that has access to your classes. The simplest way is to start it from your classes directory.
- ▶ Step four: start the service
 - ▶ Bring up another terminal and start your service
 - ▶ This might be from a main() method in your remote implementation class, or from a separate launcher class.

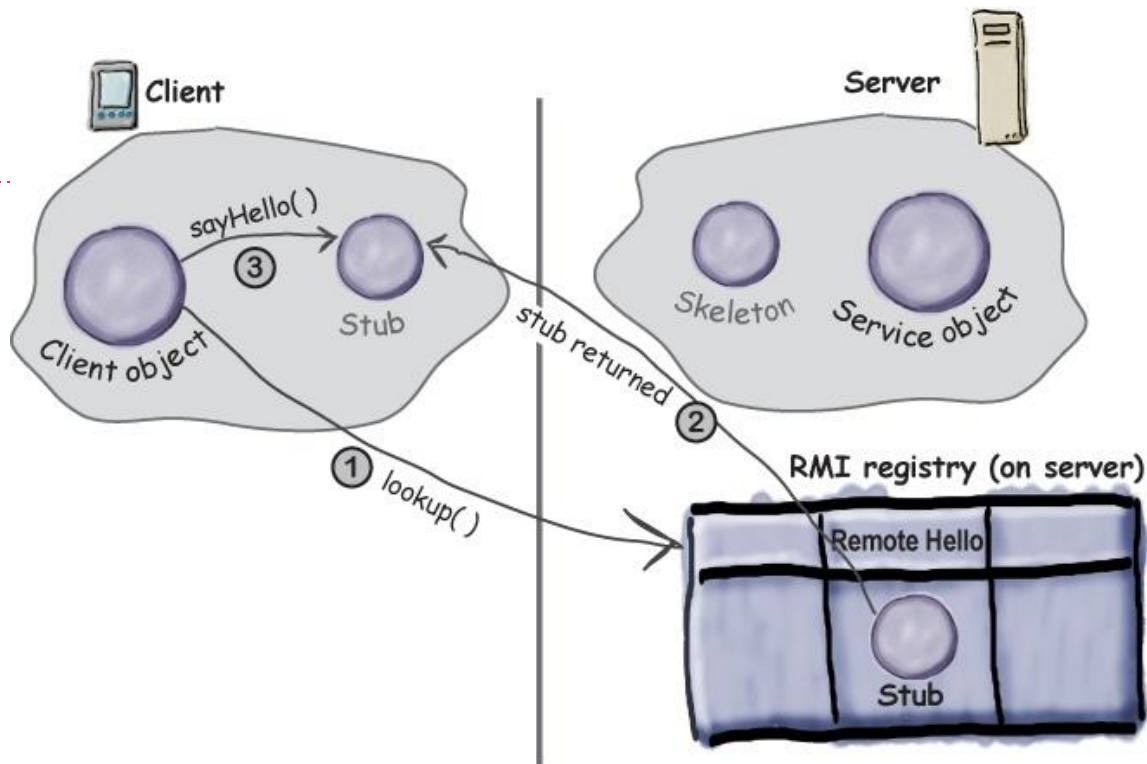
How does the client get the stub object?

- ▶ The client has to get the stub object (our proxy), since that's the thing the client will call methods on.
 - ▶ That's where the RMI registry comes in.
 - ▶ The client does a 'lookup', like going to the white pages of a phone book, and essentially says, "Here's a name, and I'd like the stub that goes with that name."

lookup() is a static method of the Naming class.

```
MyRemote service = (MyRemote)  
naming.lookup("rmi://127.0.0.1:1999/RemoteHello");
```

The client always uses the remote interface as the type of the service. In fact, the client never needs to know the actual class name of your remote service.



The Client Code

- ▶ Client does a lookup on the RMI registry
- ▶ RMI registry returns the stub object (as the return value of the lookup method) and RMI deserializes the stub automatically.
- ▶ Client invokes a method on the stub, as if the stub IS the real service

The Client Code (2)

```
import java.rmi.*;
public class MyRemoteClient {
    public static void main (String[] args) {
        new MyRemoteClient().go();
    }
    public void go() {
        try {
            MyRemote service = (MyRemote)

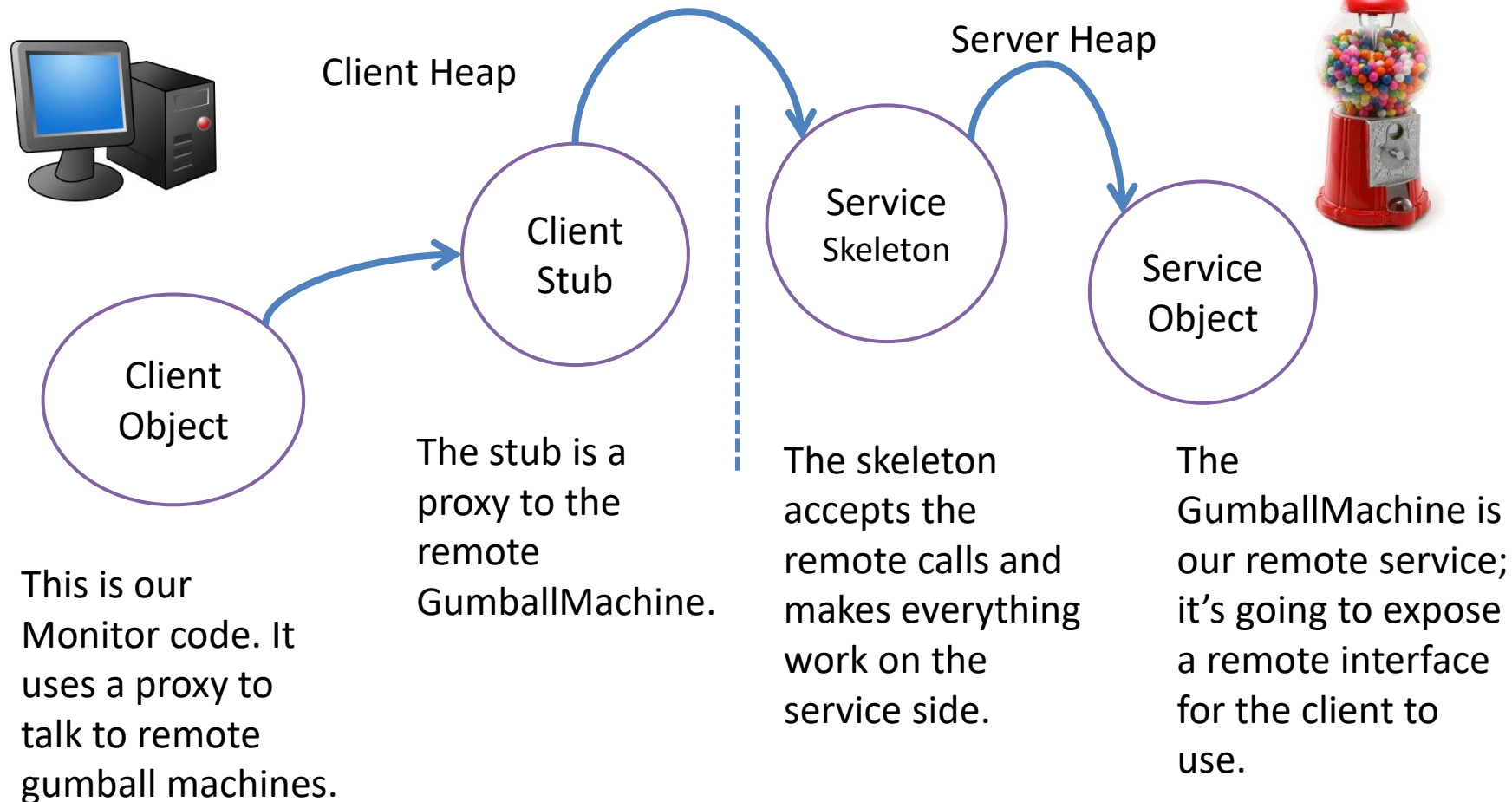
            Naming.lookup("rmi://127.0.0.1:1999/RemoteHello");
            String s = service.sayHello();
            System.out.println(s);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

The things programmers do wrong with RMI are:

1. Forget to start rmiregistry before starting remote service (when the service is registered using Naming.rebind(), the rmiregistry must be running!)
2. Forget to make arguments and return types serializable (you won't know until runtime; this is not something the compiler will detect).



The GumballMachine remote proxy



Getting the GumballMachine ready to be a remote service

- ▶ In other words, we're going to make the GumballMachine into a service.
 - ▶ 1. Create a remote interface for the GumballMachine. This will provide a set of methods that can be called remotely.
 - ▶ 2. Make sure all the return types in the interface are serializable.
 - ▶ 3. Implement the interface in a concrete class.

```
import java.rmi.*;

public interface GumballMachineRemote extends Remote {
    public int getCount() throws RemoteException;
    public String getLocation() throws RemoteException;
    public State getState() throws RemoteException;
}

import java.io.*;

public interface State extends Serializable {
    public void insertQuarter();
    public void ejectQuarter();
    public void turnCrank();
    public void dispense();
}
```

The concrete state implementations

```
public class NoQuarterState implements State {
    private static final long serialVersionUID=2L;
    transient GumballMachine gumballMachine;

    public NoQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You inserted a quarter");
        gumballMachine.setState(gumballMachine.getHasQuarterState());
    }

    public void ejectQuarter() {
        System.out.println("You haven't inserted a quarter");
    }

    public void turnCrank() {
        System.out.println("You turned, but there's no quarter");
    }

    public void dispense() {
        System.out.println("You need to pay first");
    }
}
```

The transient keyword tells the JVM not to serialize this field.

Note that this can be slightly dangerous if you try to access this field once the object's been serialized and transferred

```
import java.rmi.*;
import java.rmi.server.*;
```

GumballMachine RMI

```
public class GumballMachine
    extends UnicastRemoteObject implements GumballMachineRemote
{
    private static final long serialVersionUID = 2L;
    // other instance variables here

    public GumballMachine(String location, int numberGumballs) throws
RemoteException {
        // code here
    }

    public int getCount() {
        return count;
    }

    public State getState() {
        return state;
    }

    public String getLocation() {
        return location;
    }
    // other methods here
}
```



Registering with the RMI registry...

```
public class GumballMachineTestDrive {  
  
    public static void main(String[] args) {  
        GumballMachine gumballMachine = null;  
        int count;  
        if (args.length < 2) {  
            System.out.println("GumballMachine <name> <inventory>");  
            System.exit(1);  
        }  
        try {  
            count = Integer.parseInt(args[1]);  
            gumballMachine = new GumballMachine(args[0], count);  
            Naming.rebind("//" + args[0] + "/gumballmachine",  
gumballMachine);  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

GumballMonitor client

```
import java.rmi.*;

public class GumballMonitor {
    GumballMachineRemote machine;

    public GumballMonitor(GumballMachineRemote machine) {
        this.machine = machine;
    }

    public void report() {
        try {
            System.out.println("Gumball Machine: " + machine.getLocation());
            System.out.println("Current inventory: " + machine.getCount() + "
gumballs");
            System.out.println("Current state: " + machine.getState());
        } catch (RemoteException e) {
            e.printStackTrace();
        }
    }
}
```

We need to catch any remote exceptions that might happen as we try to invoke methods that are ultimately happening over the network.

Monitoring test

```
import java.rmi.*;

public class GumballMonitorTestDrive {

    public static void main(String[] args) {
        String[] location = {"rmi://santafe.mightygumball.com/gumballmachine",
                             "rmi://boulder.mightygumball.com/gumballmachine",
                             "rmi://seattle.mightygumball.com/gumballmachine"};

        GumballMonitor[] monitor = new GumballMonitor[location.length];

        for (int i=0; i < location.length; i++) {
            try {
                GumballMachineRemote machine =
                    (GumballMachineRemote) Naming.lookup(location[i]);
                monitor[i] = new GumballMonitor(machine);
                System.out.println(monitor[i]);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }

        for (int i=0; i < monitor.length; i++) {
            monitor[i].report();
        }
    }
}
```

This returns a proxy to the remote Gumball Machine (or throws an exception if one can't be located).

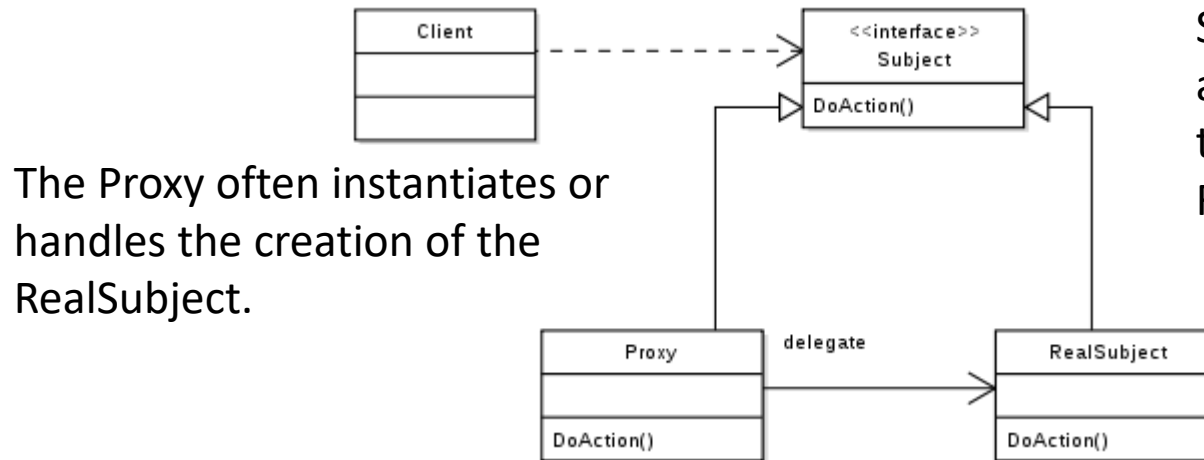
Once we get a proxy to the remote machine, we create a new GumballMonitor and pass it the machine to the monitor



The Proxy Pattern defined

- ▶ The Proxy Pattern provides a surrogate or placeholder for another object to control access to it.
- ▶ We have seen how the proxy provides a "representation" of another object
- ▶ But what about a proxy controlling access?
 - ▶ The proxy needed to control access because our client, the monitor, didn't know how to talk to a remote object.
 - ▶ So in some sense the remote proxy controlled access so that it could handle the network details for us.
 - ▶ There are many variations of the Proxy Pattern, and the variations typically revolve around the way the proxy "controls access."
 - ▶ A remote proxy controls access to a remote object.
 - ▶ A virtual proxy controls access to a resource that is expensive to create.
 - ▶ A protection proxy controls access to a resource based on access rights.

The UML Diagram



The Proxy often instantiates or handles the creation of the RealSubject.

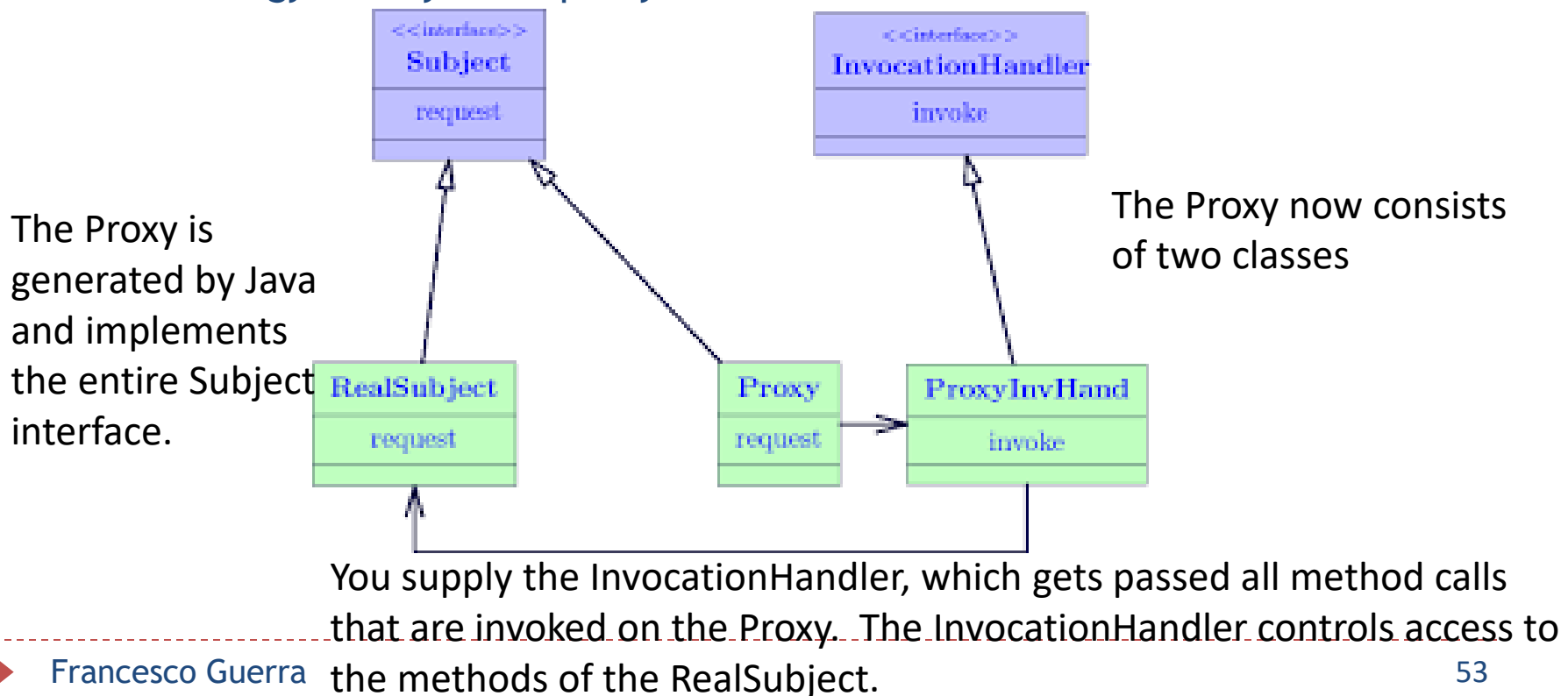
Both the Proxy and the RealSubject implement the Subject interface. This allows any client to treat the proxy just like the RealSubject

The Proxy keeps a reference to the Subject, so it can forward requests to the Subject when necessary.

The RealSubject is usually the object that does most of the real work; the Proxy controls access to it.

Using the Java API's Proxy to create a protection proxy

- ▶ Java's got its own proxy support right in the `java.lang.reflect` package.
 - ▶ Java lets you create a proxy class on the fly that implements one or more interfaces and forwards method invocations to a class that you specify.
 - ▶ Because the actual proxy class is created at runtime, we refer to this Java technology as a dynamic proxy.



Using the Java API's Proxy to create a protection proxy (2)

- ▶ Java creates the Proxy class for you, you need a way to tell the Proxy class what to do.
 - ▶ You can't put that code into the Proxy class like we did before, because you're not implementing one directly.
 - ▶ The job of the InvocationHandler is to respond to any method calls on the proxy.
 - ▶ Think of the InvocationHandler as the object the Proxy asks to do all the real work after it's received the method calls.

Example

- We want to create an application for rating the professors

```
public class ProfessorBeanImpl implements ProfessorBean {
    String name;
    String gender;
    String topic;
    int rating;
    int ratingCount = 0;

    public String getName() {
        return name;
    }

    public int rating() {
        if (ratingCount == 0) return 0;
        return (rating/ratingCount);
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setRating(int rating) {
        this.rating += rating;
        ratingCount++;
    }
}

public interface ProfessorBean {

    String getName();
    String getGender();
    String getTopic();
    int getRating();

    void setName(String name);
    void setGender(String gender);
    void setTopic(String interests);
    void setRating(int rating);
}
```

Some setters and getters
are not reported

Keeping the software protected

- ▶ You shouldn't be able to vote for yourself or to change another customer's data.
 - ▶ The way our ProfessorBean is defined, any client can call any of the methods.
- ▶ This is a perfect example of where we might be able to use a Protection Proxy.
 - ▶ Protection Proxy is a proxy that controls access based on access rights.
 - ▶ For instance, if we had a student object, a Protection Proxy might allow the student to call certain methods on the object, a Rector to call additional methods (like `setSalary()`), and a human resources employee to call any method on the object.

Creating a Dynamic Proxy

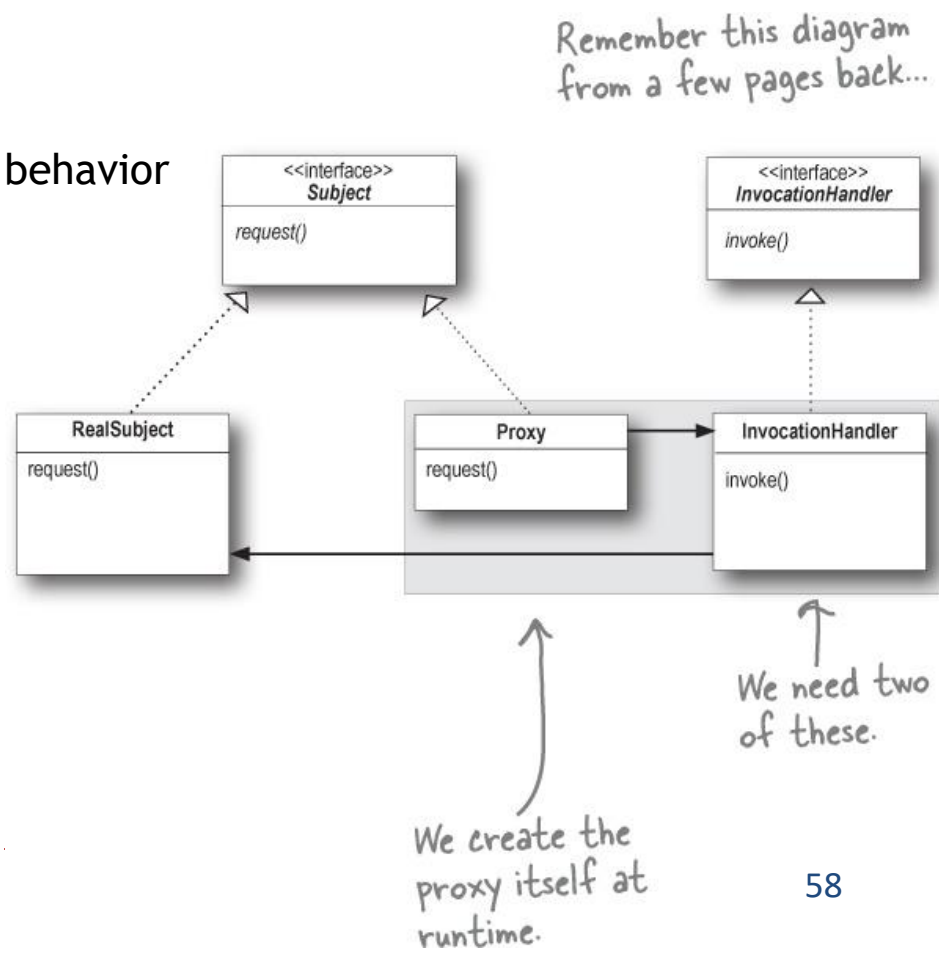
- ▶ In our service
 - ▶ we want to make sure that a Professor can set his own information while preventing others from altering it.
 - ▶ we want to allow just the opposite with the ratings: we want the other Professors to be able to set the rating, but not that particular Professor.
- ▶ To fix these problems we're going to create two proxies:
 - ▶ one for accessing your own ProfessorBean object
 - ▶ one for accessing another user's ProfessorBean object.
- ▶ That way, the proxies can control what requests can be made in each circumstance.

Java Dynamic Proxy

- ▶ Java creates two proxies
- ▶ We need to supply the handlers that know what to do when a method is invoked on the proxy.

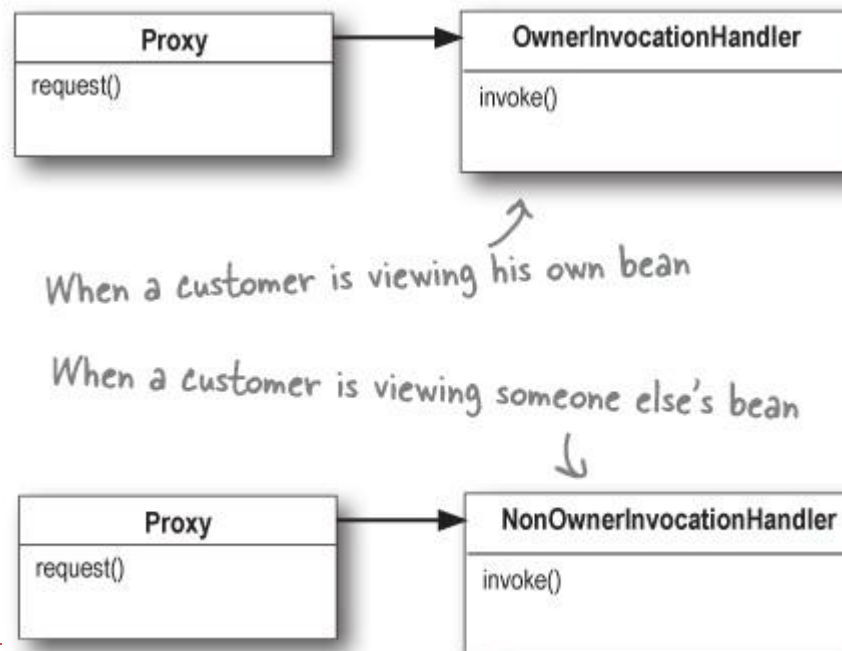
- ▶ Create two `InvocationHandlers`.

- ▶ `InvocationHandlers` implement the behavior of the proxy.
- ▶ Java will take care of creating the actual proxy class and object;
- ▶ we just need to supply a handler that knows what to do when a method is called on it.



Java Dynamic Proxy(2)

- ▶ Write the code that creates the dynamic proxies.
 - ▶ We need to write a little bit of code to generate the proxy class and instantiate it.
- ▶ Wrap any ProfessorBean object with the appropriate proxy.
 - ▶ When we need to use a ProfessorBean object, either it's the object of the customer himself, or it's another user.



Creating Invocation Handlers

- ▶ We need to write two invocation handlers,
 - ▶ one for the owner one for the non-owner.
- ▶ What are invocation handlers?
 - ▶ When a method call is made on the proxy, the proxy forwards that call to your invocation handler
 - ▶ There's only one method, `invoke()`, and no matter what methods get called on the proxy, the `invoke()` method is what gets called on the handler.

Let's say the `setRating()` method is called on the proxy

```
proxy.setRating(9);
```

The proxy then turns around and calls `invoke()` on the `InvocationHandler`.

```
invoke(Object proxy, Method method, Object[] args)
```

```
return method.invoke(person, args);
```

The handler decides what it should do with the request and possibly forwards it on to the `RealSubject`.

Creating Invocation Handlers (2)

```
import java.lang.reflect.*;
```

```
public class OwnerInvocationHandler implements InvocationHandler {
    ProfessornBean person;
```

```
    public OwnerInvocationHandler(ProfessorBean person) {
        this.person = person;
    }
```

We're passed the Real Subject in the constructor and we keep a reference to it.

```
    public Object invoke(Object proxy, Method method, Object[] args)
        throws IllegalAccessException {
```

```
        try {
            if (method.getName().startsWith("get")) {
                return method.invoke(person, args);
            } else if (method.getName().equals("setRating")) {
                throw new IllegalAccessException();
            } else if (method.getName().startsWith("set")) {
                return method.invoke(person, args);
            }
        }
```

If the method is a getter, we go ahead and invoke it on the real subject.

```
    } catch (InvocationTargetException e) {
        e.printStackTrace();
    }
    return null;
}
```

If it is the setRating() method we disallow it by throwing a IllegalAccessException.



Creating Invocation Handlers (3)

```
import java.lang.reflect.*;

public class NonOwnerInvocationHandler implements InvocationHandler {
    ProfessorBean person;

    public NonOwnerInvocationHandler(ProfessorBean person) {
        this.person = person;
    }

    public Object invoke(Object proxy, Method method, Object[] args)
        throws IllegalAccessException {

        try {
            if (method.getName().startsWith("get")) {
                return method.invoke(person, args);
            } else if (method.getName().equals("setRating")) {
                return method.invoke(person, args);
            } else if (method.getName().startsWith("set")) {
                throw new IllegalAccessException();
            }
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```



Creating the Proxy class and instantiating the Proxy object

This method takes a person object (the real subject) and returns a proxy for it. Because the proxy has the same interface as the subject, we return a ProfessorBean.

```
ProfessorBean getOwnerProxy(ProfessorBean person) {  
  
    return (ProfessorBean) Proxy.newProxyInstance(  
        person.getClass().getClassLoader(),  
        person.getClass().getInterfaces(),  
        new OwnerInvocationHandler(person));  
}
```

This code creates the proxy. Now this is some mighty ugly code, so let's step through it carefully.

```
public static Object newProxyInstance(ClassLoader loader, Class<?>[] interfaces, InvocationHandler h)
```

```
public class MatchMakingTestDrive {
    // instance variables here
    public static void main(String[] args) {
        MatchMakingTestDrive test = new MatchMakingTestDrive();
        test.drive();
    }
    public void drive() {
        PersonBean joe = getPersonFromDatabase("Joe Javabean");
        PersonBean ownerProxy = getOwnerProxy(joe);
        System.out.println("Name is " + ownerProxy.getName());
        ownerProxy.setInterests("bowling, Go");
        System.out.println("Interests set from owner proxy");
        try {
            ownerProxy.setRating(10);
        } catch (Exception e) {
            System.out.println("Can't set rating from owner proxy");
        }
        System.out.println("Rating is " + ownerProxy.getRating());
        PersonBean nonOwnerProxy = getNonOwnerProxy(joe);
        System.out.println("Name is " + nonOwnerProxy.getName());
        try {
            nonOwnerProxy.setInterests("bowling, Go");
        } catch (Exception e) {
            System.out.println("Can't set interests from non owner proxy");
        }
        nonOwnerProxy.setRating(3);
        System.out.println("Rating set from non owner proxy");
        System.out.println("Rating is " + nonOwnerProxy.getRating());
    }
}
```



Key points

- ▶ The Proxy Pattern provides a representative for another object in order to control the client's access to it. There are a number of ways it can manage that access.
 - ▶ A Remote Proxy manages interaction between a client and a remote object.
 - ▶ A Virtual Proxy controls access to an object that is expensive to instantiate.
 - ▶ A Protection Proxy controls access to the methods of an object based on the caller. ▫
 - ▶ Many other variants of the Proxy Pattern exist including caching proxies, synchronization proxies, firewall proxies, copy-on-write proxies, and so on.
- ▶ Proxy is structurally similar to Decorator, but the two differ in their purpose.
 - ▶ The Decorator Pattern adds behavior to an object, while a Proxy controls access.
- ▶ Java's built-in support for Proxy can build a dynamic proxy class on demand and dispatch all calls on it to a handler of your choosing.
 - ▶ Like any wrapper, proxies will increase the number of classes and objects in your designs.

References

- ▶ Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns, O'Reilly Media 2004
- ▶ Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.